ELI — Full code review: issues, proposals and decomposition

Contents

2026-06-07. ***PLAN AND DOCUMENT ONLY — no code changes.*** *Grounded in a structural scan of the actual repository.*

0. Guiding principle (corrected)

Keep good, working, tested code. Patches and wrappers that do a real job — especially load-bearing ones — **stay**. We only act on *genuine* problems (bugs, silently-swallowed failures, dead/duplicated code, files too long to reason about). Where a god-file is split, good code is **relocated into a clearer home, never rewritten or removed.** The ~6,900-test suite is the contract for "nothing broke."

---

1. Patches & monkeypatches — reassessed: mostly KEEP

A full scan = **12 `globals()` self-patches + 8 install-time wrappers**, and the verdict is the opposite of "rip them out": almost all are good or already-retired. **Net: nothing to eliminate.** A few get a *clearer home* when their file is split — that's it.

Patch	Verdict	Why
`gguf_inference` `generate = _wrap_generate(generate)`	**KEEP as-is**	Load-bearing — `generate` self-recurses for auto-reload; a plain rebind double-wraps → `RecursionError` (reproduced). Good code doing a real job.
`gguf_inference` effective-runtime `load_model`/snapshot contract	**KEEP** (relocate with the loader)	Implements the adaptive VRAM fit + effective-runtime reporting. Works; well-tested.
`gguf_inference` `_live_runtime_params` / `_live_runtime_override` globals	**KEEP**	Module-level runtime state is a legitimate Python pattern; read in several places + the GUI. Optional future polish (a `RuntimeParams` object), **not** a priority.

Patch	Verdict	Why
`router globals()[name] = voice_contract_wrap(fn)`	**KEEP** (optional decorator form later)	Applies the voice-output contract to route callables. Works; the decorator form is cosmetic.
`executor` middleware-table / script-safety / GUI-audit install	**KEEP** (relocate into the dispatcher)	The middleware + safety contracts are real features; they just become the dispatcher's explicit list when the file is split.
`engine` non-quick persona pipeline guard	**KEEP** (becomes a named stage)	A genuine guard; clearer as an explicit pipeline stage, same behaviour.
`grounded_remediation _PENDING` global	**KEEP**	Simple pending-confirm state; works.
`memory _MemoryModule` / `_MemoryMeta` dynamic-module metaclass	**KEEP + document**	Deliberate lazy-module-attribute design, widely depended on. Not a bug — document it; only revisit if it ever blocks a split.
Already-retired stacks (v10–v17 contracts; UVRS; CognitiveEngine.process wrappers; stale router shells)	**DONE**	Prior phases (BW2 / 2c) already removed these — the precedent that good cleanup was done where warranted.

Conclusion: the patch surface is healthy. The earlier "eliminate monkeypatches" framing was wrong; these are kept. The real issues are elsewhere (§2).

---

2. Real issues, prioritized (with proposals)

P1 — Silent exception swallowing (observability) ⚠ highest-value

~**798** bare `except Exception: … pass` across the codebase (of **2,755** total `except Exception`). The logged/handled ones are fine; the **silent** ones can hide real failures — and that fights ELI's own "honest, no-fake-actions" principle. - **Proposal (nuanced — do NOT blanket-remove):** keep best-effort guards, but make them *observable*: convert silent `pass` to `log.debug(...)` (cheap, reversible, zero behaviour change). Add a **lint/claims test** that fails on *new* bare silent swallows. Triage the ~798 by module (start with `executor`, `engine`, `router`) so failures surface in logs without changing control flow.

P2 — Dead/shadowed standalone image engine — RESOLVED

The `eli/tools/image_engine/` package wins the import; the standalone module that sat beside it (1,750 LOC) was therefore unreachable dead code. **It has been deleted** — the package exports every symbol the GUI imports (`ImageGenerationRequest`, `generate_images`, `discover_local_image_models`, `discover_presets`, `list_recent_outputs`, `output_dir`), verified before removal. ~1,750 LOC removed with zero runtime effect. The internal `image_engine/image_engine/` double-nest remains (cosmetic; left as-is).

P3 — God-files (organization, not bugs) → §4

7 files hold **~58k LOC (44% of the codebase)**: executor 14.3k, engine 13.4k, GUI 11.0k, router 7.1k, labs 5.7k, memory 4.5k, grounding-gate 4.3k. They *work and are tested* — the issue is they're hard to reason about and risky to change. Decomposition plan in §4 (relocate, don't rewrite).

P4 — Dead package + root clutter

- `eli/guards/` — empty package, **0 importers → removed**.

- Root one-offs: `patch_gpu_dynamic.py`, `patch_s11_bugs.py` (applied long ago), `[package-index-options]` (junk filename), `matrix_probe_note.md`, duplicate `Install instructions` (vs `install.sh`/README), `examine_eli_safe.sh`.
- **Proposal:** move the one-offs to `archive/` (or delete after confirming they're spent); remove the junk filename; delete the empty `guards/` package. Pure hygiene.
- *(Keep:* `eli/brain/agents/` — it's the runtime custom-agent store, not dead; `scripts/` — CLI tools; `docs/`, `packaging/`, `bin/` — legitimate.)

P5 — Marker debt: essentially none

Only **1** `TODO/FIXME/HACK/XXX` marker in the whole tree — the codebase is unusually clean on this axis. No action.

P6 — The model/VRAM ceiling (context, not a code defect)

The intelligence ceiling is the local 7B on 8 GB (7B + vision + nomic + 28k KV is already near full). Not a code bug — recorded so the review is complete. Lever: a bigger GPU / model swap (the architecture is model-agnostic and ready for it).

---

3. Issue priority summary

Pri	Issue	Effort	Behaviour risk
P1	Silent `except: pass` → observable + lint guard	medium (triage)	none (logging only)
P2	Delete shadowed `image_engine.py` (1.75k LOC) + flatten nest	low	none (already unreachable)
P3	God-file decomposition (§4)	high (phased)	low (relocate + re-export)
P4	Remove `guards/` + root one-offs	low	none
P5	marker debt	—	—
P6	model ceiling	n/a (hardware)	—

---

4. God-file decomposition (relocate good code; tree + wiring)

The 174 executor handlers, 18 GUI tabs, 90 router matchers, engine pipeline stages, and memory/grounding logic all have **clean internal seams** — they relocate into coherent modules without rewriting.

4.1 Target tree

```
eli/
  execution/
   executor_enhanced.py      # thin dispatcher + back-compat re-exports (execute, SUPPORTED_ACTION
   actions/                # 17 category modules (the manifest categories) + _registry + _helpers
   routing/                # pipeline.py (the 33-stage order) + contracts.py + matchers/<domain>.py
```

```
    router_enhanced.py          # re-exports route()
  cognition/
    engine.py                   # slim CognitiveEngine orchestrator (re-exports kept)
   pipeline/             # prompt_assembly / persona / grounding / guards / verbatim + stages/
    gguf/                         # loader + the (kept) generate wrapper + effective-
runtime contract
  gui/
   eli_pro_audio_gui_MKI.py   # window shell + shared state + send_to_chat/_engine_ask + assembly
    tabs/                     # one module per main tab (chat, image, files, settings, …)
    tabs/labs/                  # one module per Labs sub-tab
  memory/
    core/                       # paths.py / store.py (metaclass kept; documented)
  runtime/
    grounding/                  # gate.py / tiers.py / signals.py
```

Every existing import path keeps working via **re-export shims** in the old module names.

4.2 Per-file

- **executor → `actions/`**: 174 handlers grouped by the 17 capability categories, each registering via an `@action("NAME")` decorator into a dispatch table; shared helpers (`_save_artifact`, evidence hook, path resolvers) → `_helpers.py`; the kept middleware/safety contracts → the registry's explicit middleware list.
- **engine → `cognition/pipeline/`**: the ~2,840 pre-class module functions (already "plain module-level functions") group into prompt/persona/grounding/guards/verbatim; `process()`'s 12 stages → `pipeline/stages/`; the persona guard → a named stage.
- **GUI → `gui/tabs/`**: one module per `create_*_tab` (pattern already started with Labs / Report Builder / Test & Review / Orchestration). Window keeps menu/toolbar + shared state.
- **router → `routing/`**: 90 matchers → domain modules; `pipeline.py` is the explicit stage order (was the install marker); voice contract → a decorator.
- **labs → `tabs/labs/`**: one module per sub-tab widget.
- **memory → `core/`**: `DBPaths`→paths, `Memory`→store; metaclass kept + documented.
- **grounding gate → `runtime/grounding/`**: tier modules.

4.3 Wiring correctness (100%)

1. **Façade re-exports** — old module paths re-export new symbols; no external import changes.
2. **Move, don't rewrite** — pure relocation first; any refactor is a separate later step.
3. **Suite as contract** — ~6,900 tests incl. claims (symbol inventory; all 208 capabilities well-formed + handled; activation phrases; blueprint refs; modules compile) run **green after every extraction**.
4. **Per-module commits**; manifest re-sync after the executor split; an import-cycle guard test (new packages never import the shim).

---

5. Sequencing (lowest-risk first; each phase green + committed)

- **Phase 1 — hygiene (P2, P4):** delete shadowed `image_engine.py` + flatten nest; remove `guards/` + root one-offs. ~1.75k LOC of dead code gone, zero behaviour risk.
- **Phase 2 — observability (P1):** silent `except: pass` → `log.debug`, module by module (executor/engine/router first) + a lint guard against new ones.
- **Phase 3 — GUI tabs (P3):** extract `tabs/` + `tabs/labs/`. Lowest-risk split.
- **Phase 4 — executor actions (P3):** category modules + registry.
- **Phase 5 — router (P3):** domain matchers + pipeline assembler.
- **Phase 6 — engine pipeline (P3):** stages + helper modules.
- **Phase 7 — gguf / grounding / memory (P3):** relocate the kept wrappers + metaclass (highest care; done last).

---

6. Preserve (must survive every step)

DAG orchestrator + agent-bus-on-DAG; evidence planner (parallel gather); report pipeline; grounding gates + anti-confabulation; self-improve→coding-agent; LoRA pipeline (model-agnostic); scheduled/overnight tasks; autonomy tick; Test & Review + Orchestration tabs; vision/gaze/voice/OS-control; netguard offline-by-default; all **208 capabilities** (live); the claims suite; and **every patch in §1 (kept)**.

7. Risks & definition of done

Risk	Mitigation
Deleting `image_engine.py` removes a needed symbol	Confirm the package exports all 6 GUI-imported names first; suite green
Logging a silent swallow changes timing	debug-level only; no control-flow change
Router/engine stage order is behavioural	pure-move; activation-phrase + mode-budget claims tests guard order
Import cycles after a split	one-way imports + a guard test

Done when: dead/duplicated code removed (image_engine.py, guards/, root one-offs); silent failures observable + a guard against new ones; no core-path file > ~2,000 LOC; all good patches kept and clearly homed; all old import paths re-export; manifest still 205; **full suite green**; a claims test forbids new god-files (>3,000 LOC) and new *silent* swallows.

Companion docs: `complete_findings.md`, `dag_orchestrator.md`, `gui.md`, `project_overview.md`, `state_snapshot.md`.